

Document number	P0475R1
Date	2018-06-05
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

LWG 2511: guaranteed copy elision for piecewise construction (rev. 1)

Revision History

Changes since P0475R0:

- Replace *UNPACK* pseudo-function with conversion to tuple of references.

Discussion

The [2511](#) issue ("[scoped_allocator_adaptor](#) piecewise construction does not require [CopyConstructible](#)") suggests removing the [CopyConstructible](#) requirement from [scoped_allocator_adaptor](#)'s piecewise construction function. I don't think I forgot to remove that requirement in my resolution for [2203](#), because the new post-2203 wording still requires copies (or moves) in some cases. However, I'm now convinced that removing the copyable requirement is important, and know how to do it.

Consider:

```
struct do_not_copy {
    do_not_copy() = default;
    do_not_copy(const do_not_copy&) { throw 1; }
};

struct X {
    using allocator_type = std::allocator<int>;
    X(do_not_copy&&, const allocator_type&) { }
};

using pair = std::pair<X, int>;
```

We can do this:

```
pair p{ std::piecewise_construct,
        std::tuple<do_not_copy, std::allocator<int>>{}},
        std::tuple<int>{} };
```

In C++17 guaranteed copy elision means this will not copy the [do_not_copy](#) in the first tuple. The [X](#) constructor takes it by reference, so there is no copy there either. (We could actually delete the [do_not_copy](#) copy constructor, but the throwing definition above allows this to work for implementations that don't support guaranteed copy elision yet).

If we try to do that with a [scoped_allocator_adaptor](#) it blows up *unless* it guarantees not to copy any of the tuple elements:

```
std::scoped_allocator_adaptor<std::allocator<pair>> a;
auto ptr = a.allocate(1);
a.construct(ptr, std::piecewise_construct,
            std::tuple<do_not_copy>{},
            std::make_tuple(1));
```

With guaranteed copy elision we can initialize the function arguments without making a copy, but if [scoped_allocator_adaptor](#) makes a copy internally by transforming the [tuple<do_not_copy>](#) into [tuple<do_not_copy, allocator<pair>>](#) then we make a copy of the [do_not_copy](#) and explode.

So without fixing LWG 2511 this doesn't work. We should fix it both for efficiency, and for consistency with guaranteed copy elision that will now happen in more places in C++17.

Simply removing the [CopyConstructible](#) requirement isn't sufficient though, because the [tuple_cat](#) operations will make copies. What's needed is to transform [tuple<Args1...>](#) into [tuple<Args1&&...>](#) or [tuple<Args1&&..., inner_allocator_type&>](#) or [tuple<allocator_arg_t, innert_allocator_type&, Args1&&...>](#) as dictated by the [uses_allocator_v](#) logic. i.e. even if the incoming tuples are not tuples of references, the ones that get passed to [pair::pair\(piecewise_construct_t, ...\)](#) should be tuples of references.

Alternative solution

Another way to ensure no copies are made would be to replace the `CopyConstructible` requirement with a requirement that `conjunction_v<is_reference_v<Args1>..., is_reference_v<Args2>...>` is true. If the incoming tuples are already tuples of references then nothing will be copied. This has the potential to break some code, whereas the proposal below doesn't.

Proposed resolution

Changes are relative to N4750.

In `[allocator.adaptor.members]`

Strike paragraph 10:

~~*Requires: all of the types in `Args1` and `Args2` shall be `CopyConstructible` (Table 22).*~~

Modify paragraph 11:

Effects: Constructs a `tuple` object `xprime` from `x` by the following rules:

- If `uses_allocator_v<T1, inner_allocator_type>` is false and `is_constructible_v<T1, Args1...>` is true, then `xprime` is `xtuple<Args1&&...>(std::move(x))`.
- Otherwise, if `uses_allocator_v<T1, inner_allocator_type>` is true and `is_constructible_v<T1, allocator_arg_t, inner_allocator_type&, Args1...>` is true, then `xprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type&>(allocator_arg, inner_allocator()), tuple<Args1&&...>(std::move(x)))`.
- Otherwise, if `uses_allocator_v<T1, inner_allocator_type>` is true and `is_constructible_v<T1, Args1..., inner_allocator_type&>` is true, then `xprime` is `tuple_cat(tuple<Args1&&...>(std::move(x)), tuple<inner_allocator_type&>(inner_allocator()))`.
- Otherwise, the program is ill-formed.

and constructs a `tuple` object `yprime` from `y` by the following rules:

- If `uses_allocator_v<T2, inner_allocator_type>` is false and `is_constructible_v<T2, Args2...>` is true, then `yprime` is `ytuple<Args2&&...>(std::move(y))`.
- Otherwise, if `uses_allocator_v<T2, inner_allocator_type>` is true and `is_constructible_v<T2, allocator_arg_t, inner_allocator_type&, Args2...>` is true, then `yprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type&>(allocator_arg, inner_allocator()), tuple<Args2&&...>(std::move(y)))`.
- Otherwise, if `uses_allocator_v<T2, inner_allocator_type>` is true and `is_constructible_v<T2, Args2..., inner_allocator_type&>` is true, then `yprime` is `tuple_cat(tuple<Args2&&...>(std::move(y)), tuple<inner_allocator_type&>(inner_allocator()))`.
- Otherwise, the program is ill-formed.

Acknowledgements

Thanks to Tim Song for suggesting the simplified specification in the R1 revision.